INTERNSHIP REPORT

Submitted

For the Partial Fulfilment of the Requirement for the Degree of Bachelor of _____ with _____ (Major and Minor Subjects) under **Magadh University**

Submitted to

The Department of _____, _____ **(College Name)**

[Paste College Logo here]

Submitted by

Name : _____

Semester : _____

Programme FYUGP : _____

University's Registration No. : _____

College Roll No. : _____

Year : _____

RK NEXORA
CAMPUS TO CAREER
A Unit of RKNexora Private Limited.

INTERNSHIP REPORT

Topic - Python Programming

Submitted to the Head of Department, _____ (College Name), in partial fulfilment of the requirements for the Degree of _____, Magadh University

Name of Student : _____

Programme : _____

University Registration No. : _____

College Roll No. : _____

Internship Registration No. : _____

Name of the IPO / Agency : RKNEXORA PRIVATE LIMITED

Internship Period : _____ to _____

Semester : _____

Year / Session : _____



_____ _____

Signature of Student **Seal & Signature of IPO**

ACKNOWLEDGEMENT

I express my sincere gratitude to RK NEXORA, RKNEXORA PRIVATE LIMITED, _____ (College Name), my faculty mentors and the internship supervisor, _____, for their guidance and support. The structured activities, learning resources and regular feedback helped me complete this internship in Python Programming successfully.

I am thankful to the Head of the Department and the college mentor for their encouragement and for guiding me in preparing this report. I am also grateful to the management of RKNEXORA PRIVATE LIMITED for providing a professional learning environment and the opportunity to learn and apply my knowledge in a practical setting. The concept sessions, hands-on tasks with sample data and case studies, and the supervisor's patient and detailed feedback helped me understand how programs are planned, written and tested step by step.

Finally, I thank my family and peers for their constant encouragement throughout the programme.

_____

Name: _____

University Registration No. - _____

INTERNSHIP COMPLETION CERTIFICATE

The Internship Completion Certificate issued by RKNEXORA PRIVATE LIMITED for the internship in Python Programming is to be attached on this page.

[Attach the original / printed copy of the QR-verified Internship Completion Certificate here]

Certificate No. _____

CONTENTS

Acknowledgement..... 3

Internship Completion Certificate..... 4

Preface..... 6

Executive Summary..... 7

About the Agency..... 8

Plan and Execution..... 9

Learning Outcome..... 32

Conclusion..... 34

Attendance Sheet..... 36

PREFACE

Internships are an important part of the undergraduate programme, as they allow students to see how classroom learning is applied in real work. The Four-Year Undergraduate Programme (FYUGP), introduced in line with the National Education Policy 2020, gives special importance to experiential learning, skill development and employability. This report has been prepared as part of the internship requirement of Magadh University and the Higher Education guidelines applicable in Bihar.

The report describes the internship undertaken in Python Programming at RKNEXORA PRIVATE LIMITED, the plan followed, the programs written and tested, and the learning gained. Almost every modern service - a mobile payment, a railway ticket, a weather forecast, a school result or a farmer's market price - depends on software, and Python is one of the easiest and most widely used languages in which such software is written. It is used in web development, data analysis, automation, artificial intelligence, science and teaching. For a student from Bihar, where many young people are preparing for careers in information technology, data and government digital services, a first-hand understanding of Python is a valuable foundation.

The report has been arranged in the order suggested by the University, beginning with the cover page and acknowledgement and ending with the conclusion and attendance sheet. Unlike a purely theoretical report, it contains the actual program code written during the internship, the real output of each program shown as terminal screenshots, a flowchart, a chart and a complete mini project with its tests. All programs were run on a computer and the outputs shown are the outputs actually produced. Only dummy data and fictitious names were used, so no real personal information has been handled.

EXECUTIVE SUMMARY

This report documents the internship completed in Python Programming at RKNEXORA PRIVATE LIMITED (RK NEXORA - 'Campus to Career'). The programme required a minimum of 120 hours, completed over about four to five weeks (28 working days) with daily sessions of four to five hours, and was supported by an attendance log, supervisor guidance and review.

Background. Python is a high-level, general-purpose programming language known for its simple, readable syntax. A program is a set of instructions that tells a computer what to do, and learning to write programs develops logical thinking and problem solving that is useful in every discipline. Python is free, runs on all common computers and has a very large collection of ready-made libraries, which is why it is used in schools, universities and industry alike.

Objectives. The internship aimed to (i) learn the basic syntax, variables, input and output of Python, (ii) understand data types and operators, (iii) write programs using decisions and loops, (iv) write reusable functions and use modules, (v) work with lists, tuples, dictionaries and sets, (vi) read and write files, (vii) apply object-oriented programming, (viii) handle errors and debug, (ix) use NumPy and Pandas for simple data work, and (x) plan, code, test and document a small project.

Work carried out. Each topic had a concept session followed by hands-on coding. In all, more than a dozen programs were written, run and corrected on a computer, and their real output was recorded as screenshots in this report. The later part of the programme was devoted to the mini project, a case study exercise, a mock practical exercise, supervisor review and preparation of this report.

Project. The work was tied together in a project titled **Student Marks Manager**, a console program of about 110 lines that stores student marks in a CSV file, calculates total, percentage and grade, and prints a class report with the class average and the topper. The project was tested with four automated unit tests, all of which passed.

Method. A blended method was followed - learn, code, run, test, document and review - supported by a daily attendance log with day-wise remarks and regular feedback from the supervisor.

Learning gained. The student gained working knowledge of Python syntax, data types, decisions and loops, functions, collections, file handling, classes and inheritance, exception handling, NumPy and Pandas, unit testing with unittest and the habit of writing clean, commented, readable code that follows the PEP 8 style guide.

Key observations. Programming is learnt by doing: every error message taught something; small programs combined into a larger one; and a program is only complete when it is tested and documented. Simple, readable code is better than clever code.

Conclusion. The internship met its objectives and contributed to the student's academic understanding, employability and confidence. The report follows the University's suggestive format: cover page, acknowledgement, certificate, contents, preface, executive summary, about the agency, plan and execution, learning outcome, conclusion and attendance sheet.

ABOUT THE AGENCY

RK NEXORA, a unit of RKNEXORA PRIVATE LIMITED, provides structured internship and skill-development programmes under the theme 'Campus to Career'. The organization focuses on academic-industry alignment, practical exposure, assessment, documentation and employability-oriented learning.

Particulars	Details
Name of Agency	RKNEXORA PRIVATE LIMITED
Programme / Unit	RK NEXORA - Campus to Career
Registered Office	C/O Mukesh Kumar, Khiri, PS Atri, Jethiyan, Atri, Gaya - 805236, Bihar, India
CIN	U85499BR2026PTC086757
Email	rknexorapvtltd@gmail.com
Website	www.rknexora.org
Internship Topic	Python Programming

1. Vision

To support students in developing practical, professional and industry-relevant capabilities.

2. Mission

To deliver structured learning experiences that connect academic knowledge with workplace expectations.

3. Activities of the Agency

1. Structured internship programmes aligned with university and higher-education requirements.
2. Skill-development learning modules, practical activities and assignments.
3. Attendance tracking, supervision, assessment and documentation.
4. Employability-oriented guidance connecting campus learning to career readiness.

4. Programme Structure

The internship programme is organised around a fixed number of learning hours, completed over a defined period. Each intern is registered on the internship portal with a unique registration number, is assigned a supervisor, follows a day-wise plan and maintains an attendance log. Learning is organised into three phases: **Orientation** (Days 1-2), **Learning and Practice** (Days 3-22, ten sub-topics of two days each: one concept day and one hands-on day) and **Project and Review** (Days 23-28), followed by a concluding review.

5. Role of the Supervisor

The internship supervisor assigned by the agency guided the student through the programme, explained concepts, assigned practice tasks, reviewed the daily log and gave feedback. The supervisor also verified attendance and reviewed the student's work at the end of the programme.

PLAN AND EXECUTION

1. Objectives

The internship was planned around the following objectives, each linked to a part of the programme:

1. Understand Python Programming - what a program is, why Python is popular, how it runs, and how to install and use it (Weeks 1-2).
2. Learn syntax, variables, input and output, data types and operators (Week 1).
3. Learn to control the flow of a program with if-else decisions and for and while loops (Week 2).
4. Learn to write functions and use modules, and to store data in lists, tuples, dictionaries and sets (Weeks 2-3).
5. Learn file handling, object-oriented programming and error handling (Week 3-4).
6. Learn the basics of NumPy and Pandas, and plan, code, test and document a mini project (Weeks 4-5).
7. Develop professional discipline - clean code, comments, documentation, testing and report writing (throughout).

2. Methodology

The programme followed a blended method made up of four connected steps, repeated for each topic:

Learn. Each topic began with a concept session in which the supervisor explained the idea, its terms and a small example. Notes were prepared in the student's own words.

Code. The concepts were then applied by typing programs from scratch - not copying - and running them. Programs were first written for small problems such as a percentage calculator, a grade finder, a multiplication table, a marks dictionary and a CSV reader, and later combined into a larger project. Only sample data and fictitious names were used.

Test and debug. Every program was run with normal inputs and with wrong inputs, error messages were read carefully and the code was corrected. A simple rule was followed: change one thing at a time and run again.

Document and review. Every day's work was recorded in the internship attendance log with a short remark. Programs were saved with comments, and the supervisor reviewed the work, pointed out errors and suggested improvements. A case study and a mock practical exercise consolidated the learning.

3. Understanding Python Programming - Concepts Studied

3.1 What is Python and why is it used?

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. 'High-level' means that the language is close to human language and hides machine details; 'interpreted' means that programs are translated and run line by line, so a result can be seen quickly; and 'general-purpose' means that it is not limited to one kind of work. Python uses indentation (spaces at the start of a line) to show the structure of a program, which forces programmers to write tidy code.

The main reasons for the popularity of Python are its simple syntax, which makes it easy for beginners; its huge standard library and third-party packages; its free and open-source nature; its portability across Windows, Linux and macOS; and its strong community support. The main areas of use are listed below.

- **Web development** - websites and services built with frameworks such as Django and Flask.
- **Data analysis and science** - NumPy, Pandas and Matplotlib for working with numbers and drawing charts.
- **Artificial intelligence and machine learning** - libraries such as scikit-learn, TensorFlow and PyTorch.

- **Automation and scripting** - renaming files, sending reports, reading spreadsheets and web pages automatically.
- **Education and research** - Python is the first language taught in many schools and universities.
- **Software testing and DevOps** - writing automated tests and tools for servers.

3.2 How a Python program runs

A Python program is written in a text file with the extension .py. When it is run, Python first compiles the source code into an intermediate form called bytecode, and the Python Virtual Machine (PVM) then executes the bytecode line by line and produces the output. If there is a mistake in the code, Python stops and displays an error message with the line number, which helps the programmer find the problem. This flow is shown in the figure below.

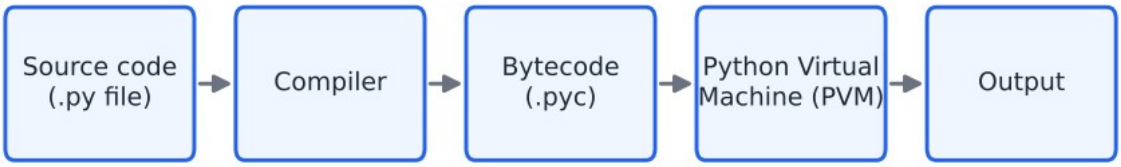


Figure 1: How a Python program is executed

3.3 Setting up the environment

The student installed Python from the official website, checked the installation from the terminal and used a code editor (IDLE and Visual Studio Code were both tried) to write programs. Packages such as NumPy and Pandas were installed using the pip tool. The commands used to check the setup and the output obtained on the practice computer are shown below. A first line of code, print('Hello, Python!'), confirms that everything works.

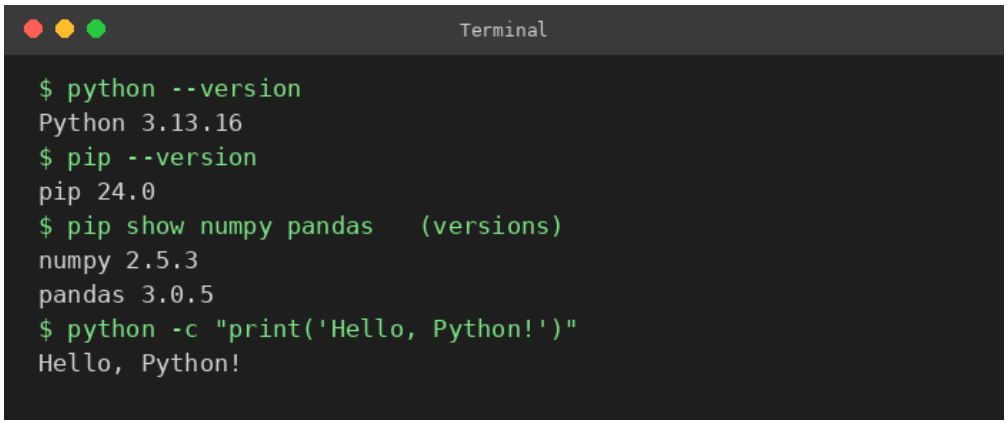


Figure 2: Checking the Python installation and the first program

Programs were saved in a separate folder for each topic. Comments, written after the # sign, were used to explain the purpose of the code, and the editor's colour highlighting and automatic indentation helped to avoid typing mistakes.

4. Week-wise Plan and Execution

Week	Days	Activities
Week 1	Days 1-6	Orientation and foundations: company induction, introduction to Python, syntax, variables, input and output, data types and operators, each with a study session and a practice task.

Week	Days	Activities
Week 2	Days 7-12	Logic building: if-else statements, for and while loops, functions, parameters, return values and modules.
Week 3	Days 13-18	Data structures and files: lists, tuples, dictionaries and sets; reading and writing text and CSV files; classes and objects.
Week 4	Days 19-24	Advanced topics: inheritance, error handling and debugging, NumPy and Pandas, and the Student Marks Manager mini project.
Week 5	Days 25-28	Project closure: case study exercise, mock practical exercise, supervisor review and feedback, internship report drafting, presentation preparation and final review.

5. Week-wise Narrative

Week 1: The first week was devoted to orientation and foundations. After company induction, the student studied what Python is, installed it, wrote the first programs using input and print, and learnt variables, numbers, strings, booleans and operators. By the end of the week the student could write a small program that asks for input, calculates a result and displays it.

Week 2: In the second week the programs began to make decisions and repeat actions. The student learnt if, elif and else, for and while loops, and wrote a grade finder and multiplication tables. The week then moved to functions - defining them, passing parameters, returning values and using the math module.

Week 3: The third week focused on collections - lists, tuples, dictionaries and sets - and on file handling, where the student saved data to a CSV file and read it back. The first class, Student, was also written in this week.

Week 4: In the fourth week the student studied inheritance, exception handling and debugging, and then used NumPy and Pandas to calculate totals and averages from a small table. The remaining days were devoted to planning, coding and testing the Student Marks Manager project.

Week 5: The last week was devoted to the case study exercise, the mock practical exercise, supervisor review and feedback, drafting of the internship report, preparation of a presentation and final review before submission.

6. Topic-wise Learning in Detail

6.1 Python Basics and Syntax

Concept: Installing Python, syntax, variables, input and output, and comments. A variable is a name that refers to a value stored in memory; input() reads text typed by the user and print() displays results. input() always returns a string, so int() or float() is used to convert it to a number. **Practice carried out:** A program that reads a student's name and marks and prints the percentage.

```
name = input("Enter your name: ")
marks = int(input("Enter your marks (out of 100): "))
percent = marks / 100 * 100
print("Hello,", name)
print("Your percentage is", percent, "%")
```

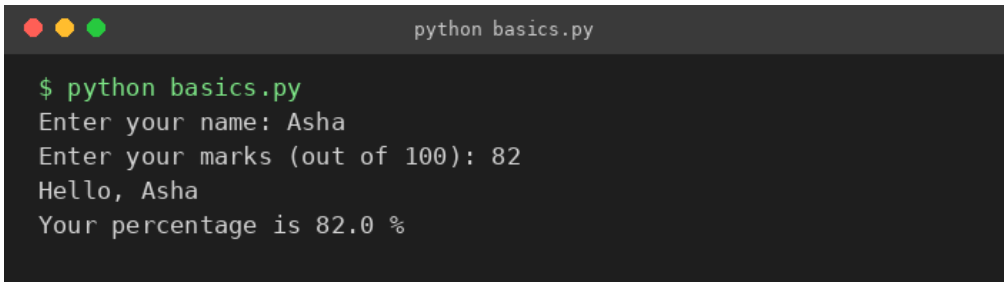


Figure 3: Output of basics.py (input is echoed as typed)

Here int(input(...)) converts the typed marks to a whole number, and the division returns a decimal value, so the percentage appears as 82.0. The program uses meaningful variable names such as name, marks and percent, as recommended by the PEP 8 style guide.

6.2 Data Types and Operators

Concept: Numbers (int, float), strings (str), booleans (bool) and operators. Python decides the type of a variable from the value assigned to it, and the type() function shows it. Arithmetic operators are `+`, `-`, `*`, `/`, `//` (floor division), `%` (remainder) and `**` (power); comparison operators are ==, `!=`, >, <, >= and <=; and logical operators are `and`, `or` and `not`. Strings can be indexed and sliced. **Practice carried out:** A program that shows the type of each kind of value and tries every operator.

```
age = 19
height = 5.6
city = "Gaya"
is_student = True
print(type(age), type(height), type(city), type(is_student))
a, b = 7, 2
print(a / b, a // b, a % b, a ** b)
word = "Python"
print(word.upper(), word[0:3], len(word))
print(10 > 5 and 3 < 1, not (10 < 5))
```

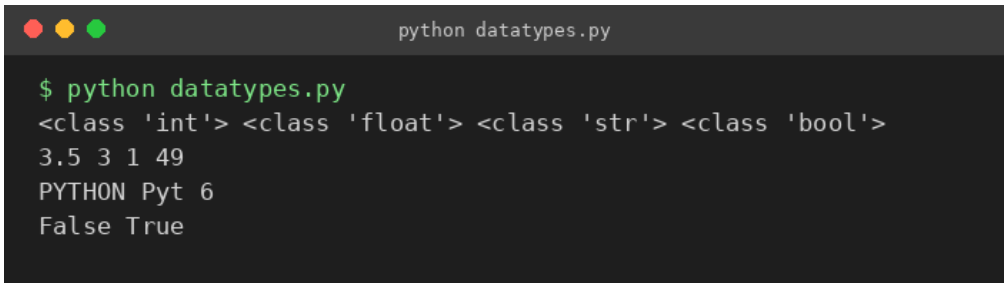


Figure 4: Output of datatypes.py

The result of 7 / 2 is 3.5, 7 // 2 is 3, 7 % 2 is 1 and 7 ** 2 is 49. For the string 'Python', upper() gives PYTHON, the slice [0:3] gives Pyt and len() gives 6. The table below summarises the operators studied.

Operator	Meaning	Example	Result
/	True division	7 / 2	3.5
//	Floor division	7 // 2	3
%	Remainder	7 % 2	1
	Power	7 ** 2	49
and / or / not	Logical operators	10 > 5 and 3 < 1	False
== / !=	Equal / not equal	5 == 5	True

6.3 Control Flow and Loops

Concept: if-elif-else statements, for loops and while loops. A decision lets the program choose between actions, and a loop repeats an action. range(1, 6) produces the numbers 1 to 5. A while loop repeats as long as its condition is true, so the condition must change inside the loop or the program will never stop. **Practice carried out:** A grade finder using if-elif-else, a multiplication table using for, and a countdown using while.

```
def grade(m):
    if m >= 75:
        return "A"
    elif m >= 60:
        return "B"
    elif m >= 40:
        return "C"
    else:
        return "Fail"

for m in [82, 67, 45, 31]:
    print(m, "->", grade(m))

print("Table of 5")
for i in range(1, 6):
    print("5 x", i, "=", 5 * i)

count = 3
while count > 0:
    print("Countdown:", count)
    count -= 1
```

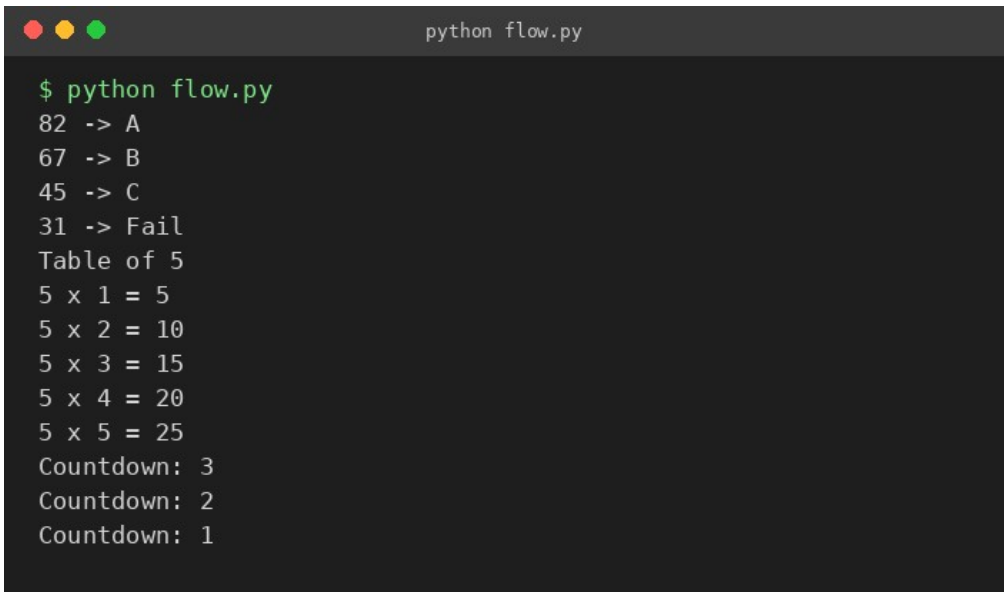


Figure 5: Output of flow.py

The marks 82, 67, 45 and 31 gave grades A, B, C and Fail, since the conditions are checked from top to bottom and the first true one is used. Placing the highest limit first is therefore important. The break and continue statements, used to leave a loop early or skip one turn, were also practised.

6.4 Functions and Modules

Concept: A function is a named block of code that can be reused. It is defined with def, takes parameters and may return a value. Default parameters (exp=2), variable arguments (*nums) and the return statement were studied. A module is a file of ready-made code; the standard math module provides sqrt, pi and other tools, and import brings it into the program. **Practice carried out:** Functions to find the area of a circle, to find the minimum, maximum and average of any number of values, and to raise a number to a power.

```
import math

def area_circle(r):
    return round(math.pi * r * r, 2)

def stats(*nums):
    return min(nums), max(nums), sum(nums) / len(nums)

def power(base, exp=2):
    return base ** exp

print(area_circle(5))
lo, hi, avg = stats(12, 9, 21, 18)
print("Min:", lo, "Max:", hi, "Average:", avg)
print(power(5), power(2, 5))
```

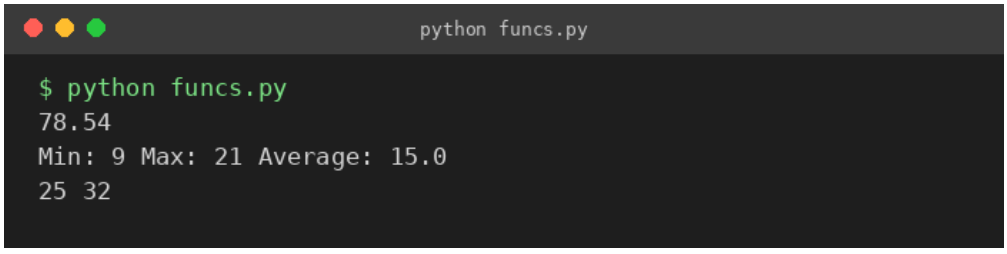


Figure 6: Output of funcs.py

The area of a circle of radius 5 is 78.54. The function stats() returns three values at once as a tuple, which are unpacked into lo, hi and avg. power(5) uses the default exponent 2 and gives 25, while power(2, 5) gives 32. Writing small functions made programs shorter, easier to test and easier to read.

6.5 Lists, Tuples, Dictionaries and Sets

Concept: A list is an ordered, changeable collection; a tuple is ordered but unchangeable; a dictionary stores key-value pairs; and a set stores unique values. Indexing starts from 0. Common list methods are append(), sort(), insert(), remove() and pop(). Dictionaries are searched by key and are ideal for records such as name and marks. **Practice carried out:** A fruit list, a tuple unpacked into two variables, a marks dictionary with ranking, and a set that removes duplicates.

```
fruits = ["mango", "banana", "litchi"]
fruits.append("guava")
fruits.sort()
print(fruits, fruits[0], len(fruits))

point = (3, 4)
x, y = point
print("x =", x, "y =", y)

marks = {"Asha": 82, "Ravi": 67, "Neha": 91}
marks["Amit"] = 45
for k, v in marks.items():
    print(k, v)
ranking = sorted(marks, key=marks.get, reverse=True)
print("Ranking:", ranking)

unique = set([1, 2, 2, 3, 3, 3])
print(unique)
```

```
python collections_demo.py

$ python collections_demo.py
['banana', 'guava', 'litchi', 'mango'] banana 4
x = 3 y = 4
Asha 82
Ravi 67
Neha 91
Amit 45
Ranking: ['Neha', 'Asha', 'Ravi', 'Amit']
{1, 2, 3}
```

Figure 7: Output of collections_demo.py

sorted(marks, key=marks.get, reverse=True) arranges the names by marks from the highest, giving Neha, Asha, Ravi and Amit. The set {1, 2, 2, 3, 3, 3} keeps only the values 1, 2 and 3. The table below compares the four collections.

Collection	Brackets	Ordered	Changeable	Duplicates	Typical use
list	[ ]	Yes	Yes	Allowed	Marks of a class
tuple	( )	Yes	No	Allowed	Fixed pair such as (x, y)
dict	{k: v}	Yes	Yes	Keys unique	Name and marks record
set	{ }	No	Yes	Not allowed	Removing duplicates

6.6 File Handling

Concept: Reading and writing text and CSV files. A file is opened with open(), and the with statement closes it automatically. Mode 'w' writes, 'r' reads and 'a' appends. The csv module reads and writes comma-separated values, which can also be opened in MS Excel. **Practice carried out:** Writing a small CSV file of roll number, name and marks and reading it back as records.

```python
import csv

rows = [["roll", "name", "marks"], [1, "Asha", 82], [2, "Ravi", 67]]
with open("sample.csv", "w", newline="") as f:
    csv.writer(f).writerows(rows)

with open("sample.csv") as f:
    for row in csv.DictReader(f):
        print(row["roll"], row["name"], row["marks"])
```

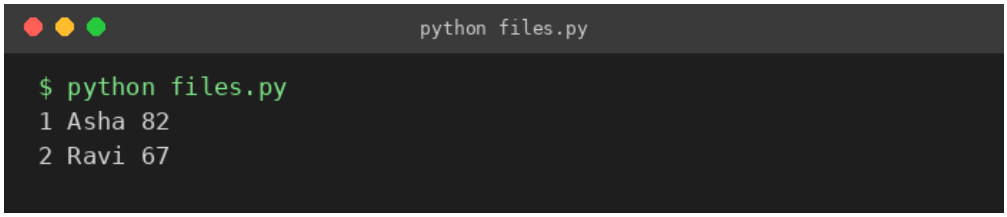


Figure 8: Output of files.py

csv.writer(f).writerows(rows) writes all rows at once, and csv.DictReader reads each row as a dictionary so that values are accessed by column name such as row['name']. Saving data in a file means it is not lost when the program closes, and this idea was used in the mini project.

6.7 Object-Oriented Programming

Concept: A class is a blueprint, and an object is one thing made from it. The __init__ method sets up the object's data (attributes), and other methods define its behaviour. Inheritance lets a new class (Topper) reuse and extend another (Student). The special method __str__ decides how an object is printed. **Practice carried out:** A Student class with a result method and a Topper class that inherits from it.

```python
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def result(self):
        return "Pass" if self.marks >= 40 else "Fail"

    def __str__(self):
        return f"I am {self.name} | Result: {self.result()}"

class Topper(Student):
    def badge(self):
        return self.marks >= 90

s = Student("Asha", 82)
print(s)
t = Topper("Neha", 91)
print(t)
print(t.badge())
```

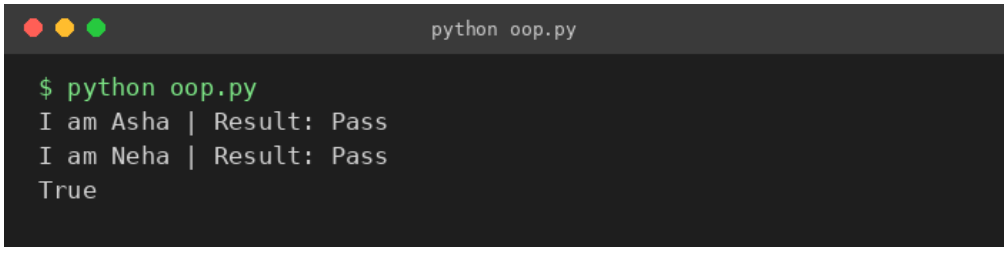


Figure 9: Output of oop.py

print(s) calls __str__ and shows 'I am Asha | Result: Pass'. The Topper object also has the result() method because of inheritance, and its own badge() method returns True for marks of 90 or more. Using a class keeps the data and the functions that work on it together, which is how the mini project was organised.

6.8 Error Handling and Debugging

Concept: Errors are of three kinds - syntax errors (wrong grammar, found before running), runtime errors or exceptions (found while running, such as dividing by zero) and logical errors (the program runs but the answer is wrong). The try-except-finally statement handles exceptions so the program does not crash. Debugging means finding and removing errors, using print statements, reading the traceback and testing with small inputs. **Practice carried out:** A safe divide function and a conversion that fails.

```
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print("Error: cannot divide by zero")
        return None
    finally:
        print("divide() finished")

print(divide(10, 2))
print(divide(5, 0))
try:
    int("abc")
except ValueError as e:
    print("ValueError:", e)
```

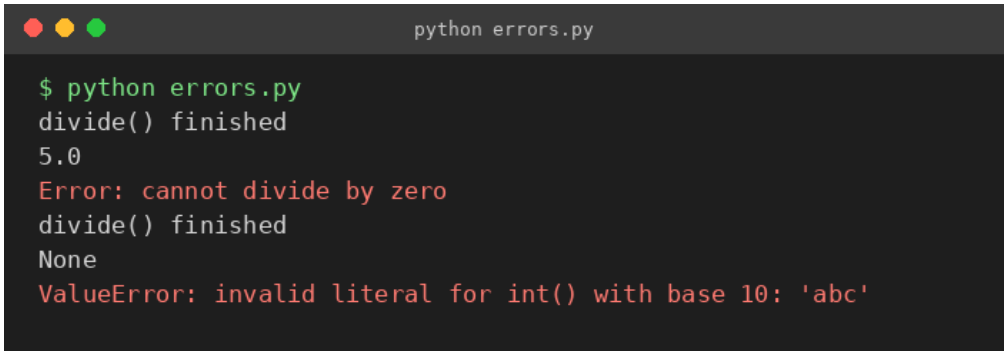


Figure 10: Output of errors.py

The finally block runs in every case, so 'divide() finished' is printed both times. Dividing 5 by 0 is caught by except ZeroDivisionError, and int('abc') raises ValueError, whose message is printed instead of crashing the program. The common errors met during practice are listed in section 9.

6.9 NumPy and Pandas

Concept: NumPy provides fast arrays, which allow an operation such as arr * 2 to be applied to every element without a loop. Pandas provides the DataFrame, a table with named columns, and makes it easy to add columns, calculate averages and filter rows. **Practice carried out:** A table of four students with three subjects, a Total column, the class mean and a filter for students with a total above 200.

```python
import math, random, datetime
import numpy as np
import pandas as pd

print(math.sqrt(49), round(math.pi, 3))
arr = np.array([1, 2, 3])
print(arr * 2, arr.mean())
df = pd.DataFrame({"Name": ["Asha", "Ravi", "Neha", "Amit"],
                   "Maths": [78, 65, 90, 52],
                   "Science": [82, 60, 88, 41],
                   "English": [86, 70, 95, 44]})
df["Total"] = df[["Maths", "Science", "English"]].sum(axis=1)
print(df)
print("Class mean (Maths):", df["Maths"].mean())
print(df[df["Total"] > 200]["Name"].tolist())
```

```
python libs.py
$ python libs.py
7.0 3.142
[2 4 6] 2.0
   Name  Maths  Science  English  Total
0  Asha     78       82       86    246
1  Ravi     65       60       70    195
2  Neha     90       88       95    273
3  Amit     52       41       44    137
Class mean (Maths): 71.25
['Asha', 'Neha']
```

Figure 11: Output of libs.py

The Total column was created with df[[...]].sum(axis=1), the mean mathematics mark was 71.25, and the filter df[df['Total'] > 200] returned Asha and Neha. A task that would take many lines with loops took only a few lines with Pandas, which shows why it is widely used in data work.

6.10 Comprehensions, Lambda and String Handling

Concept: A list comprehension builds a list in one line, a lambda is a small unnamed function, and filter() keeps the items for which a function is true. A dictionary can count how often each word occurs. **Practice carried out:** A word-frequency counter, a palindrome checker and a list of squares with its even values.

```python
text = "to be or not to be that is the question to be"
freq = {}
for w in text.split():
    freq[w] = freq.get(w, 0) + 1
top = sorted(freq.items(), key=lambda kv: (-kv[1], kv[0]))[:3]
print(top)

def is_palindrome(s):
    s = "".join(c.lower() for c in s if c.isalnum())
    return s == s[::-1]

print(is_palindrome("Madam"), is_palindrome("Python"))
squares = [n * n for n in range(1, 6)]
evens = list(filter(lambda n: n % 2 == 0, squares))
print(squares, evens)
```

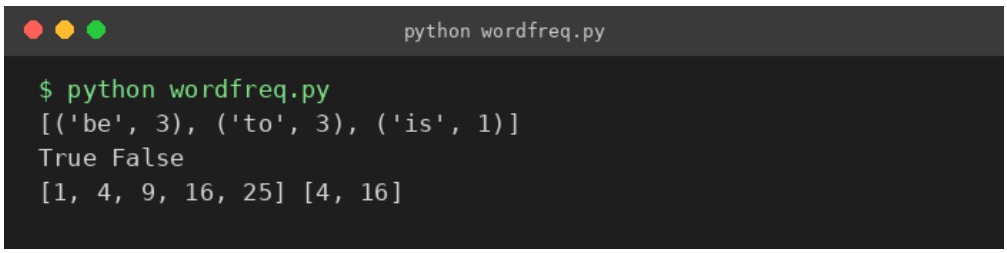


Figure 12: Output of wordfreq.py

The words 'be' and 'to' each occur three times in the sample sentence. 'Madam' is a palindrome once spaces and capital letters are ignored, while 'Python' is not. The squares of 1 to 5 are 1, 4, 9, 16 and 25, and filter() keeps the even ones, 4 and 16.

7. Good Practice, Style and Ethics - Points Studied

Writing code that works is only the first step; the code must also be readable, safe and honest. The points studied are summarised below.

- **PEP 8 style:** four spaces for indentation, lowercase names with underscores for variables and functions (total_marks), CapWords for classes (Student), spaces around operators and lines of reasonable length.
- **Meaningful names and comments:** a name such as percentage tells more than p; docstrings (triple-quoted text under a function or class) explain what the code does.
- **One job per function:** small functions are easier to test and reuse than one long block of code.
- **Input validation:** never trust what the user types; check the range and type and handle wrong input with try-except, as done in read_marks().
- **Virtual environments and requirements:** a separate environment for each project keeps the installed packages in order.
- **Version control:** saving versions of code with Git helps to go back when something breaks and to work in a team.
- **Academic honesty and licences:** code taken from a book or website should be understood, rewritten in one's own way and acknowledged; open-source licences must be respected.
- **Data privacy:** real personal data of people should not be used in practice programs or shared; the Digital Personal Data Protection Act, 2023 gives importance to the safe handling of personal data. Only dummy data was used in this internship.

8. Process Flows Studied

Programming follows a clear method. Two process flows used throughout the internship are given below.

8.1 Program development cycle

- **Understand the problem:** write what is given (input), what is wanted (output) and the rules.
- **Plan:** write the steps in simple words or draw a flowchart; choose the data structures.
- **Code:** write the program in small parts, with comments and clear names.
- **Run and test:** try normal, boundary and wrong inputs; compare the output with a hand calculation.
- **Debug:** read the error message, locate the line, fix one thing and run again.
- **Document and improve:** add comments, save the final version and note possible improvements.

8.2 Debugging checklist flow

- Read the last line of the error message first - it names the error type.
- Look at the line number given in the traceback and the line just above it.
- Print the value of the variables near the error to see what they contain.

- Check indentation, brackets, quotes and the colon at the end of if, for, while and def lines.
- Test the smallest part of the code that shows the problem, then fix it and run the whole program again.

9. Worked Examples with Sample (Dummy) Data

9.1 EMI calculator

A small program was written to calculate the monthly instalment (EMI) of a loan using the formula EMI = P x r x (1 + r)^n / ((1 + r)^n - 1), where P is the loan amount, r is the monthly rate (annual rate / 12 / 100) and n is the number of months. For a loan of Rs. 5,00,000 at 10% for 5 years, the program gives the following result. The answer was cross-checked with a hand calculation and with a spreadsheet, and the three values agreed.

```
def emi(principal, annual_rate, years):
    r = annual_rate / 12 / 100
    n = years * 12
    return principal * r * (1 + r) ** n / ((1 + r) ** n - 1)

p = float(input("Loan amount (Rs): "))
rate = float(input("Annual interest rate (%): "))
yrs = int(input("Years: "))
m = emi(p, rate, yrs)
print(f"Monthly EMI : Rs. {m:,.2f}")
print(f"Total paid  : Rs. {m * yrs * 12:,.2f}")
print(f"Interest    : Rs. {m * yrs * 12 - p:,.2f}")
```

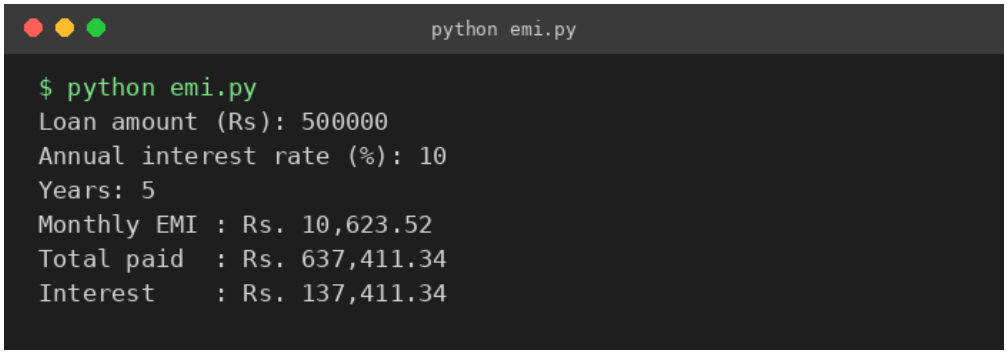


Figure 13: Output of emi.py

The monthly EMI is Rs. 10,623.52, the total repayment is Rs. 6,37,411.34 and the interest is Rs. 1,37,411.34. The exercise showed how a business formula is turned into a function and how f-strings with format codes such as :,.2f display money neatly.

9.2 Common errors met and how they were corrected

Error	Example that caused it	Reason	Correction
SyntaxError	if marks > 40 (colon missing)	Wrong grammar	Add the colon and run again
IndentationError	Body of a function not indented	Python uses spaces to show blocks	Indent by four spaces
NameError	print(totl)	Variable name misspelt or not defined	Correct the spelling
TypeError	'Age: ' + 19	Adding text and number	Use str(19) or an f-string
ValueError	int('abc')	Text cannot become a number	Validate input with try-except
ZeroDivisionError	10 / 0	Division by zero	Check the divisor first
IndexError	fruits[10] (only 4 items)	Position outside the list	Use len() or a loop
KeyError	marks['Sita']	Key not in dictionary	Use marks.get('Sita')
FileNotFoundError	open('data.csv') when missing	Wrong name or folder	Check the path or create the file

10. Project: Student Marks Manager

10.1 Problem statement and requirements

In many coaching centres and small schools, marks are written in registers and the total, percentage and grade are calculated by hand, which takes time and invites mistakes. The aim of the project was to build a small, easy-to-use console program that stores the marks of students, calculates the results automatically and prints a class report. The requirements were as follows:

- Add a student with name and marks in three subjects (Maths, Science and English).
- Accept only whole numbers from 0 to 100 and ask again for wrong input instead of crashing.
- Calculate total, percentage (rounded to two decimals) and grade: A for 75% and above, B for 60% and above, C for 40% and above, otherwise Fail.
- Save the records in a CSV file (marks.csv) so that data is not lost when the program is closed, and load it again at the start.
- Show a class report with the class average and the name of the topper.
- Provide a simple menu and handle an invalid menu choice.

10.2 Design

The program was designed around one class, Student, which holds the roll number, name and marks and knows how to calculate its own total, percentage and grade. Separate functions handle loading and saving the file, reading valid marks, adding a student and showing the report, and the main() function runs the menu in a loop. The structure and the flow of the program are shown in the figures below.

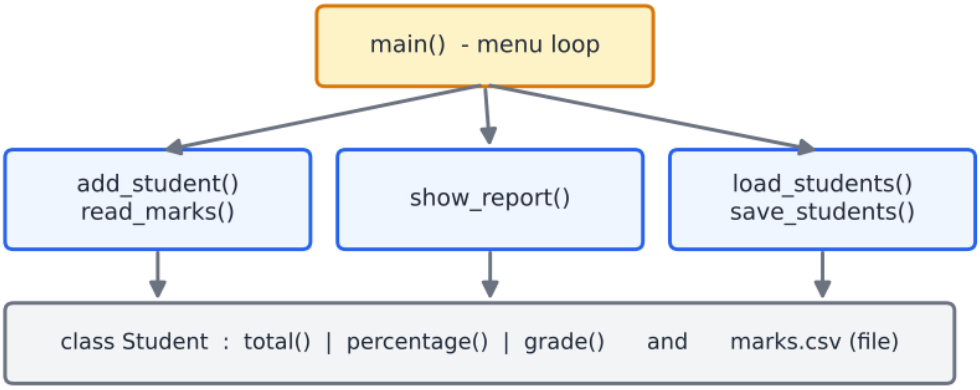
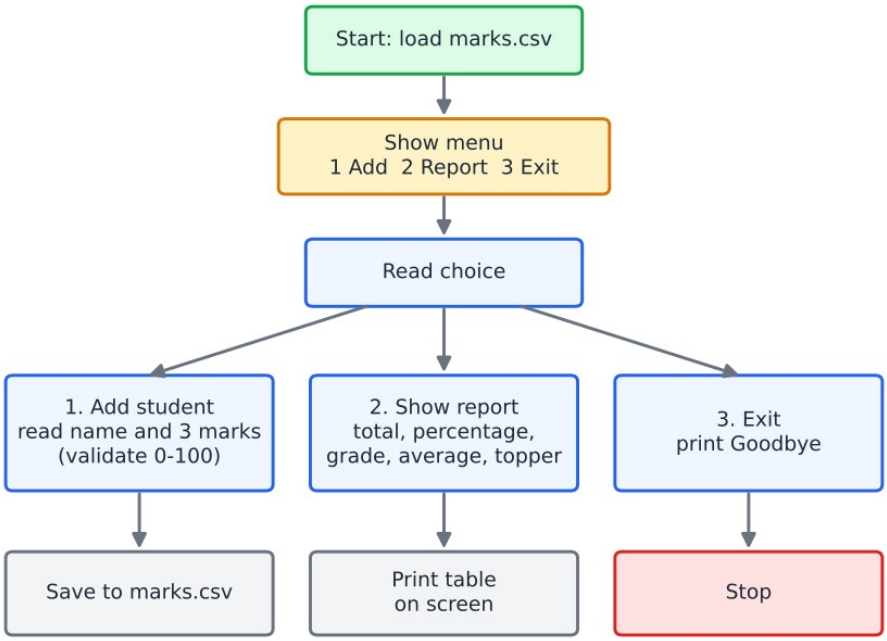


Figure 14: Structure of the project - functions, class and file



After Add or Report, the program returns to the menu (loop). Exit ends the loop.

Figure 15: Flowchart of the Student Marks Manager

10.3 Implementation - source code

The complete program (about 110 lines) is shown below as it was written in the code editor, in four parts. The Student class and the file functions come first, followed by the input and report functions and the menu in main().

```
"""Student Marks Manager - a console-based mini project.

Stores student marks in a CSV file, calculates total, percentage and grade,
and prints a class report.
"""
import csv
import os

FILE_NAME = "marks.csv"
SUBJECTS = ["Maths", "Science", "English"]


class Student:
    """Represents one student and the marks in each subject."""

    def __init__(self, roll, name, marks):
        self.roll = roll
        self.name = name
        self.marks = marks          # list of three integers

    def total(self):
        return sum(self.marks)

    def percentage(self):
        return round(self.total() / (100 * len(self.marks)) * 100, 2)

    def grade(self):
        p = self.percentage()
```

Figure 16: student_manager.py - lines 1 to 28 (imports and the Student class)

```
        if p >= 75:
            return "A"
        elif p >= 60:
            return "B"
        elif p >= 40:
            return "C"
        return "Fail"


def load_students():
    students = []
    if not os.path.exists(FILE_NAME):
        return students
    with open(FILE_NAME, newline="") as f:
        for row in csv.DictReader(f):
            marks = [int(row[s]) for s in SUBJECTS]
            students.append(Student(int(row["roll"]), row["name"], marks))
    return students


def save_students(students):
    with open(FILE_NAME, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["roll", "name"] + SUBJECTS)
        for s in students:
            writer.writerow([s.roll, s.name] + s.marks)


def read_marks(subject):
    while True:
```

Figure 17: student_manager.py - lines 29 to 58 (loading, saving and reading marks)

```
        try:
            m = int(input(f"  Marks in {subject} (0-100): "))
            if 0 <= m <= 100:
                return m
            print("  Marks must be between 0 and 100.")
        except ValueError:
            print("  Please enter a whole number.")


def add_student(students):
    name = input("Student name: ").strip()
    roll = len(students) + 1
    marks = [read_marks(s) for s in SUBJECTS]
    students.append(Student(roll, name, marks))
    save_students(students)
    print(f"Saved: {name} (Roll {roll})")


def show_report(students):
    if not students:
        print("No records found.")
        return
    print(f"{'Roll':<5}{'Name':<10}{'Total':>6}{'Percent':>9}  Grade")
    print("-" * 38)
    for s in students:
        print(f"{s.roll:<5}{s.name:<10}{s.total():>6}"
              f"{s.percentage():>9}  {s.grade()}")
    avg = sum(s.percentage() for s in students) / len(students)
```

Figure 18: student_manager.py - lines 59 to 86 (adding a student and the report)

```
87      topper = max(students, key=lambda s: s.percentage())
88      print("-" * 38)
89      print(f"Class average: {avg:.2f}% | Topper: {topper.name}")
90
91
92  def main():
93      students = load_students()
94      while True:
95          print("\n=== STUDENT MARKS MANAGER ===")
96          print("1. Add student   2. Show report   3. Exit")
97          choice = input("Enter choice: ").strip()
98          if choice == "1":
99              add_student(students)
100         elif choice == "2":
101             show_report(students)
102         elif choice == "3":
103             print("Goodbye!")
104             break
105         else:
106             print("Invalid choice. Try again.")
107
108
109 if __name__ == "__main__":
110     main()
```

Figure 19: student_manager.py - lines 87 to 110 (the menu and program start)

A few points in the code deserve mention. The condition `if __name__ == "__main__":` allows the file to be run as a program and also to be imported by the test file without starting the menu. The round() function limits the percentage to two decimals. The lambda in max(students, key=lambda s: s.percentage()) finds the topper, and the while True loop in read_marks() keeps asking until a valid mark is entered.

10.4 Running the program

The program was run with four students. During the entry of Ravi's marks the text 'abc' was typed by mistake in the English subject; the program displayed a message and asked again instead of stopping. The first part of the run is shown below.

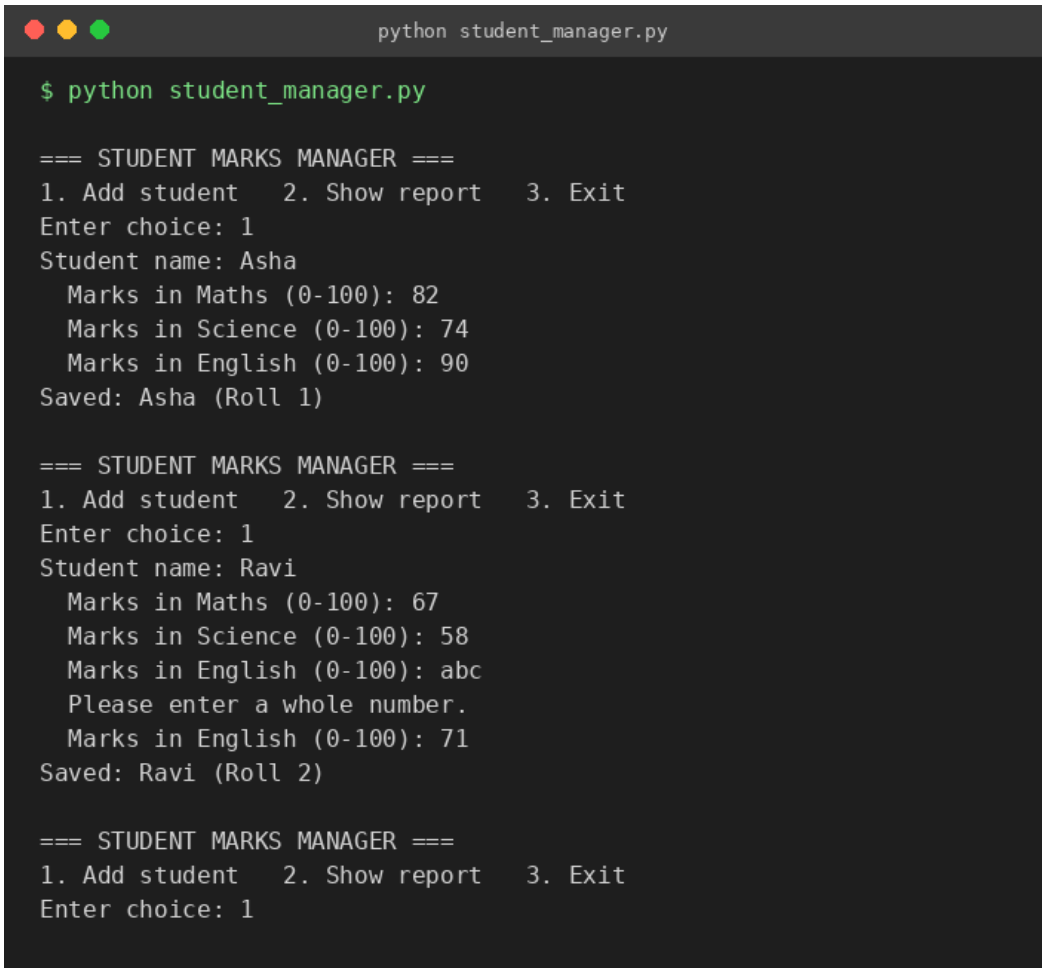


Figure 20: Adding students (note the handling of the wrong entry 'abc')

After all four students were added, option 2 was chosen to display the class report, and then an invalid choice (5) and the exit option (3) were tried. The data was saved in marks.csv, whose first lines are: roll,name,Maths,Science,English followed by 1,Asha,82,74,90 and so on.

```
python student_manager.py
$ python student_manager.py   (report and exit)

=== STUDENT MARKS MANAGER ===
1. Add student   2. Show report   3. Exit
Enter choice: 2
Roll Name       Total  Percent  Grade
--------------------------------------
1    Asha         246     82.0  A
2    Ravi         196    65.33  B
3    Neha         274    91.33  A
4    Amit         135     45.0  C
--------------------------------------
Class average: 70.91% | Topper: Neha

=== STUDENT MARKS MANAGER ===
1. Add student   2. Show report   3. Exit
Enter choice: 5
Invalid choice. Try again.

=== STUDENT MARKS MANAGER ===
1. Add student   2. Show report   3. Exit
Enter choice: 3
Goodbye!
```

Figure 21: Class report, invalid choice and exit

10.5 Results

The report shows that Asha scored 246 marks (82.0%, grade A), Ravi 196 (65.33%, B), Neha 274 (91.33%, A) and Amit 135 (45.0%, C). The class average was 70.91% and the topper was Neha. The percentages are shown in the chart below, which was drawn with the Matplotlib library from the same results.

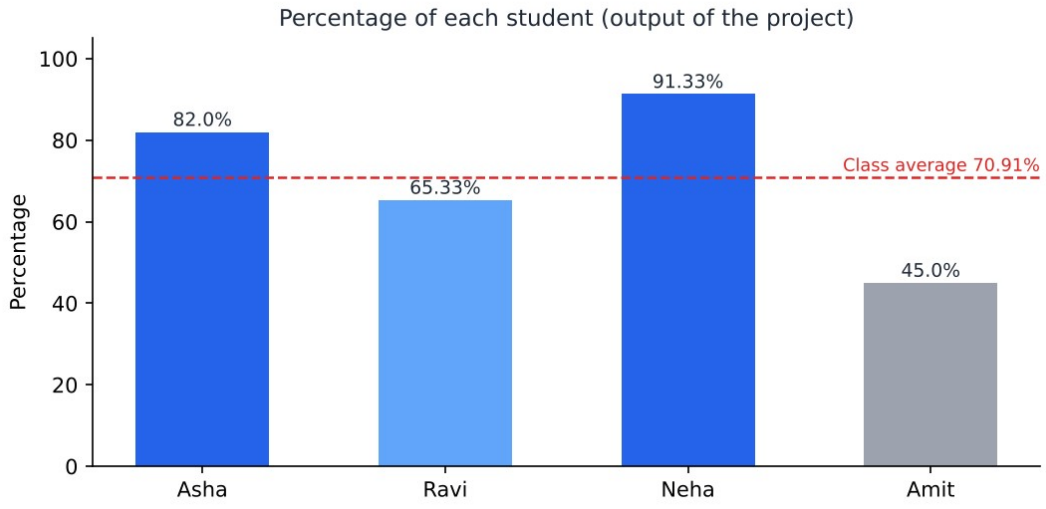


Figure 22: Percentage of each student and the class average

10.6 Testing

Testing is part of programming, not an extra. Four unit tests were written with Python's built-in unittest module. Each test creates a Student object with known marks and checks that the result is exactly what is expected by hand calculation: the total of 82, 74 and 90 is 246, the percentage is 82.0, the grade is A, and a student with 30, 35 and 20 marks gets Fail.

```python
import unittest
from student_manager import Student


class TestStudent(unittest.TestCase):
    def setUp(self):
        self.s = Student(1, "Asha", [82, 74, 90])

    def test_total(self):
        self.assertEqual(self.s.total(), 246)

    def test_percentage(self):
        self.assertEqual(self.s.percentage(), 82.0)

    def test_grade_a(self):
        self.assertEqual(self.s.grade(), "A")

    def test_grade_fail(self):
        self.assertEqual(Student(2, "Amit", [30, 35, 20]).grade(), "Fail")


if __name__ == "__main__":
    unittest.main()
```

```
python -m unittest -v test_student

$ python -m unittest -v test_student
test_grade_a (test_student.TestStudent.test_grade_a) ... ok
test_grade_fail (test_student.TestStudent.test_grade_fail) ... ok
test_percentage (test_student.TestStudent.test_percentage) ... ok
test_total (test_student.TestStudent.test_total) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```

Figure 23: Result of the unit tests - 4 tests passed

All four tests passed. Besides the automated tests, the program was tested manually with wrong inputs (text instead of a number, marks above 100, a wrong menu choice and an empty file), and it handled each of them without crashing.

10.7 Limitations and future improvements

- The program has a text menu only; a graphical interface (Tkinter) or a web page (Flask) could be added.
- Students cannot yet be edited or deleted; update and delete options can be added.
- The subject names are fixed in the code; they could be read from a settings file.
- Pandas could be used to sort the report, draw charts automatically and export an Excel file.
- More tests can be written for the file functions and the validation.

11. Case Study Exercise: Automating the Marks Register of a Coaching Centre

A fictitious coaching centre in Gaya has about 120 students and four teachers. After every weekly test, each teacher writes marks in a register, and the centre assistant spends nearly two evenings copying the marks into a sheet, adding totals by hand and preparing a result slip for each student. Mistakes are common: totals are sometimes wrong, a student's marks are entered against the wrong roll number, and parents complain that results come late. The case was given to the student for analysis.

Problem analysis. The main problem is not the lack of effort but a manual process that repeats the same calculation again and again. The causes identified were duplicate entry of data, no checking of marks, no single place where data is stored and no standard format for results.

Proposed Python solution. The marks would be entered once into a CSV file (or directly through a program like the Student Marks Manager). A Python script would validate that every mark lies between 0 and 100, calculate totals, percentages and grades, list the toppers and the students who need help, and print or save a result slip for each student. The same data could then be analysed with Pandas to find the average of each subject and the questions on which the whole class did badly.

Expected benefits and cautions. The work of two evenings would be reduced to a few minutes, calculation mistakes would almost disappear, and teachers could spend the time saved on teaching. The cautions noted were to keep a backup of the file, to protect students' personal information, to test the program on old data before using it and to keep the teachers trained so that the process does not depend on one person.

12. Mock Practical Exercise: Building and Testing a Small Program

A mock practical exercise was conducted at the end of the learning phase to check that the student could build a complete program independently in a fixed time. The task was to write a program for a small shop that reads the name, price and quantity of items, calculates the bill with 5% GST, and saves the bill to a file. The steps followed were:

1. Understanding the task: the inputs, outputs and the rule for GST were written down in simple words.
2. Planning: a list of steps and the variables needed were prepared; a dictionary was chosen to hold item, price and quantity.
3. Coding: a function to calculate the line total, a loop to read items until the user types 'done', and a function to print the bill.
4. Validation: price and quantity were checked to be positive numbers using try-except.
5. Saving: the bill was written to a text file using the with statement.
6. Testing: the program was run with three items, with a wrong price and with no items; results were compared with a hand calculation.
7. Review: the supervisor checked the naming, comments and structure, and suggested splitting the code into smaller functions, which was done.
8. Presentation: the program and its output were explained to the supervisor in about five minutes.

13. Sample Checklists Prepared

Code quality checklist

- Variable and function names are meaningful and follow PEP 8.
- Every function does one job and has a short comment or docstring.
- No repeated code that could be moved into a function.
- Wrong input is handled without crashing the program.
- Indentation is consistent (four spaces).

Testing checklist

- Normal input gives the expected output (checked by hand).
- Boundary values such as 0, 40, 75 and 100 were tried.
- Wrong input such as text, empty values and negative numbers was tried.
- The program works again after closing and reopening (file saved and loaded).
- Unit tests were run and all passed.

Submission checklist

- Programs saved in separate folders with clear file names.
- Output screenshots taken from the real run of the program.
- Sample data is dummy; no personal data of real people is included.
- Attendance log, certificate and report are complete and signed.

14. Tools and Resources

The following tools and resources were used during the internship: Python 3 with IDLE and Visual Studio Code, the pip package installer, the NumPy, Pandas and Matplotlib libraries, the built-in unittest module for testing, MS Excel for checking calculations and opening CSV files, MS Word for the report, the official Python documentation (docs.python.org), learning materials and practice questions given by the agency, and regular mentor feedback.

15. Programme at a Glance

The programme was in the domain of Python Programming with RKNEXORA PRIVATE LIMITED as the agency. It required a minimum of 120 hours over 28 learning days, organised in three phases: Orientation (Days 1-2), Learning and Practice (Days 3-22) and Project and Review (Days 23-28). Attendance was recorded in the Internship Attendance Log, signed by the supervisor (see Attendance Sheet). The approximate distribution of the effort across the topics is shown below.

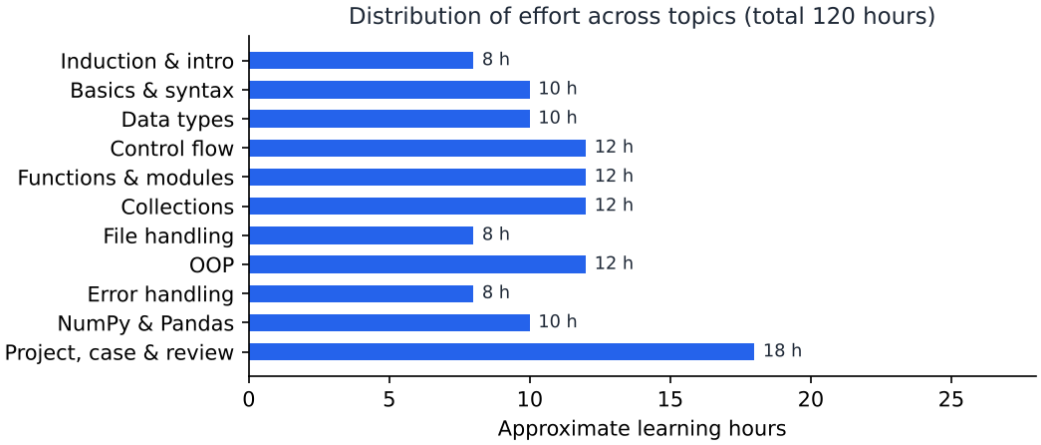


Figure 24: Distribution of learning effort across topics (120 hours)

LEARNING OUTCOME

a. Knowledge Acquired

1. What Python is, how a program is compiled to bytecode and run by the Python Virtual Machine, and how to install Python and run programs.
2. Variables, input and output, numbers, strings, booleans, type conversion and the arithmetic, comparison and logical operators.
3. Decision making with if-elif-else and repetition with for and while loops, including range, break and continue.
4. Writing functions with parameters, default values and return values, and using modules such as math.
5. Lists, tuples, dictionaries and sets, with indexing, slicing, sorting and iteration.
6. Reading and writing text and CSV files with the with statement and the csv module.
7. Object-oriented programming: classes, objects, attributes, methods, inheritance and the __str__ method.
8. Types of errors, try-except-finally, debugging with tracebacks, and unit testing with unittest.
9. Basic data work with NumPy arrays and Pandas DataFrames, and drawing a chart with Matplotlib.
10. Planning, coding, testing and documenting a complete mini project, following PEP 8 style and good practice.

Glossary of Terms Learned

- **Program** - A set of instructions that a computer follows.
- **Interpreter / Compiler** - Tools that translate code so that the computer can run it.
- **Bytecode / PVM** - Intermediate code of Python / the machine that runs it.
- **Variable** - A name that refers to a value stored in memory.
- **Data type** - The kind of value: int, float, str, bool, list, tuple, dict, set.
- **Operator** - A symbol that performs an action on values, such as + or ==.
- **Indentation** - Spaces at the start of a line that show which code belongs to a block.
- **Loop** - A statement that repeats code (for, while).
- **Function** - A named, reusable block of code that may take parameters and return a value.
- **Parameter / Argument** - The name in the definition / the value passed in the call.
- **Module / Package / Library** - A file of code / a folder of modules / a collection of ready-made tools.
- **List comprehension** - A short way to build a list in one line.
- **Dictionary** - A collection of key-value pairs.
- **Class / Object** - A blueprint / a thing made from it.
- **Inheritance** - A new class reusing the features of another class.
- **Exception** - An error found while the program runs.
- **Traceback** - The message that shows where an error happened.
- **Debugging** - Finding and fixing errors in code.
- **Unit test** - A small automatic test that checks one piece of code.
- **CSV** - A text file of values separated by commas.
- **DataFrame** - A table of rows and named columns in Pandas.
- **PEP 8** - The official style guide for writing Python code.

b. Skills Learned

Technical skills. Writing and running Python programs; using variables, operators, decisions and loops; writing functions and modules; working with lists, dictionaries and sets; reading and writing CSV files; designing classes and using inheritance; handling exceptions and debugging; using NumPy, Pandas and Matplotlib for simple data work; writing unit tests; and documenting code with comments and docstrings.

Professional skills. Logical and structured thinking; problem solving; attention to detail; patience in finding errors; time management; clear documentation and report writing; presenting a program and explaining its working; and openness to feedback.

These skills were developed through repeated practice and daily documentation. Typing programs from scratch built accuracy, reading error messages built patience and logic, testing built the habit of checking results, and explaining the project to the supervisor built confidence and communication.

c. Observed Attitude and Gained Values

1. **Patience and persistence:** most programs failed at first; steady debugging led to success.
2. **Honesty:** code was written and understood by the student, and sources were acknowledged.
3. **Attention to detail:** a single missing colon or wrong indentation can stop a program, so careful checking became a habit.
4. **Responsibility for data:** only dummy data was used and the importance of data privacy was understood.
5. **Discipline and punctuality:** regular reporting at fixed timings and daily attendance records.

d. Challenges Faced

Understanding indentation and syntax errors in the first week was handled by typing programs carefully, using the editor's highlighting and reading the error line number.

Understanding the difference between a list, tuple, dictionary and set was handled by making a comparison table and writing a small program for each.

Understanding classes and the use of self was difficult at first; it was handled by making a simple Student class and printing its objects step by step.

Finding logical errors, where the program runs but gives a wrong answer, was handled by printing intermediate values and testing with hand-calculated examples.

Making the project handle wrong input was handled by using try-except inside a loop, as in read_marks().

Managing time between coding, documentation and report writing was handled by dividing work week by week and using review days for corrections.

e. Career Pathways

The knowledge gained supports careers such as Python Developer (junior), Software Trainee, Automation and Testing Executive, Data Entry and Data Analyst Assistant, Junior Data Analyst, Web Developer (with Django or Flask), Technical Support Executive and, with further study, Data Scientist or Machine Learning Engineer. Opportunities exist in IT companies, banks, start-ups, government digital projects, education and research. Related higher-study options include BCA, MCA, B.Tech and M.Tech in computer science and information technology, M.Sc. in data science, and certificate courses in Python, SQL, data analytics and artificial intelligence. A good next step is to practise on a coding platform, learn SQL and Git, build two or three small projects and keep them on a public code-sharing site as a portfolio.

CONCLUSION

The internship in Python Programming was a valuable learning experience that integrated academic knowledge with practical coding. The programme achieved its objectives through structured learning, guided practice, assignments, supervisor review and documentation.

The student gained a clear understanding of how programs are planned, written and tested - through variables and data types, decisions and loops, functions, collections, files, classes, error handling and libraries - and acquired practical experience by building and testing the Student Marks Manager. The experience has strengthened the student's employability and provides a foundation for further study and work in software development, data analysis and automation.

Reflection

Looking back, the most useful lesson of the internship was that programming is learnt by doing and by making mistakes. Every error message, from a missing colon to a wrong index, pointed to something that could be understood and corrected. Small programs that worked became the building blocks of a larger one, and the project showed how a real problem - a manual marks register - can be turned into a working solution. Testing gave confidence that the answers were correct, and the habit of recording each day's work made progress visible over the weeks.

Objectives and Achievement

The objective of learning syntax, data types and operators was achieved through the first programs and the operator table. The objective of writing programs with decisions, loops and functions was achieved through the grade finder, tables and the EMI calculator. The objective of using collections and files was achieved through the marks dictionary and the CSV programs. The objective of object-oriented programming and error handling was achieved through the Student class and the safe divide program. The objective of using NumPy and Pandas was achieved through the marks table analysis, and the objective of planning, coding, testing and documenting a project was achieved through the Student Marks Manager and its four passing unit tests.

Future Scope

The knowledge gained can support further study of computer science and data science and careers in software, automation, testing and data analysis. The project can be extended into a web application or a desktop program with a database. With the growth of digital services, e-governance and data-driven decisions in Bihar and across India, people who can write clear programs and work with data are increasingly needed.

Suggestions for Improvement of the Programme

1. Provide a short introduction to Git and a code-sharing site so that students can keep their projects online.
2. Include a small group project and a code-review session in which students read and improve each other's programs.
3. Add practice with SQL and a simple web framework such as Flask.
4. Continue regular mentor feedback and progress reviews.
5. Encourage students to maintain detailed daily documentation.

References

1. Kolb, D. A. (1984). Experiential Learning: Experience as the Source of Learning and Development. Prentice Hall.
2. University Grants Commission (UGC). Internship guidelines for higher education institutions.
3. Ministry of Education, Government of India. National Education Policy 2020.

4. Python Software Foundation. The Python Tutorial and Python Standard Library documentation. docs.python.org.
5. van Rossum, G., Warsaw, B. and Coghlan, A. PEP 8 - Style Guide for Python Code. peps.python.org.
6. Sweigart, A. Automate the Boring Stuff with Python. No Starch Press.
7. Matthes, E. Python Crash Course. No Starch Press.
8. McKinney, W. Python for Data Analysis. O'Reilly Media.
9. NumPy and Pandas project documentation. numpy.org and pandas.pydata.org.
10. Government of India. The Digital Personal Data Protection Act, 2023.
11. RK NEXORA internship programme handbook.

ATTENDANCE SHEET

The Internship Attendance Log issued by the IPO, with the supervisor's signature for each day, is to be attached on this page.

[Attach the signed Internship Attendance Log here]